

设计模式报告：观察者模式在论坛消息通知中的应用

3230105934 公衍之

一、问题描述

在论坛系统中，用户发帖或回复后，通常需要触发多项后续操作，比如发送消息通知、统计帖子热度、记录操作日志等。

如果直接在发帖方法里依次调用这些功能模块，会导致业务逻辑和通知、统计、日志等模块耦合过紧。以后每新增一种处理逻辑，都需要去改发帖的代码，维护起来会很麻烦。

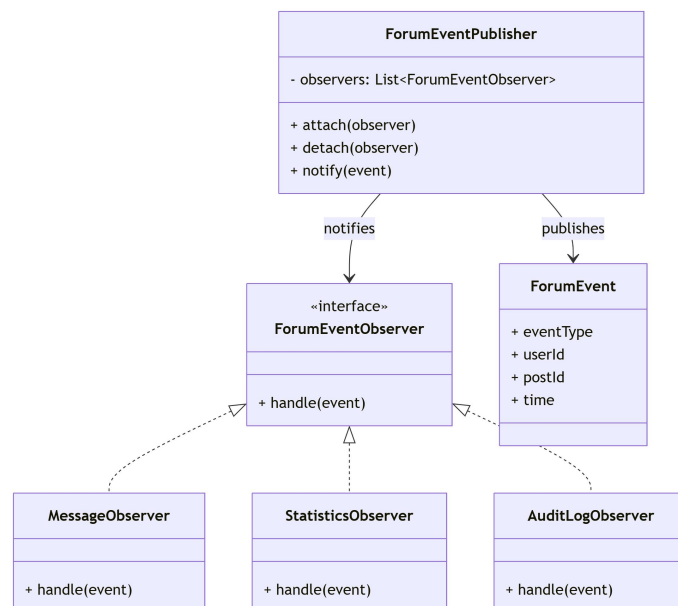
二、设计模式选择

本情境可以采用观察者模式（Observer）。观察者模式定义了对象之间的一对多的依赖关系，当一个对象状态发生变化时，所有依赖它的对象都会收到通知并自动更新。在这个场景里，论坛事件发布者是 Subject，消息通知模块、统计模块、日志模块是 Observer。当发帖或回复事件发生时，发布者统一通知所有观察者，由各观察者完成自己的处理逻辑。这样一来，“触发动作”与“具体业务处理”的逻辑就可以较好地分离。

三、角色设计

- ForumEventPublisher：事件发布者，负责注册、移除、通知观察者
- ForumEventObserver：观察者接口，定义统一的事件处理方法
- MessageObserver：消息通知观察者，负责发送消息提醒
- StatisticsObserver：统计观察者，负责更新帖子热度或用户活跃度
- AuditLogObserver：日志观察者，负责记录用户操作日志
- ForumEvent：事件对象，封装事件类型、用户 ID、帖子 ID、时间等信息

四、UML 类图



五、设计说明

当用户在论坛发布帖子或回复后，发帖业务模块不直接调用消息、统计、日志模块，而是做以下步骤：

- 1、生成一个 ForumEvent 对象 e，里面保存事件类型、用户 ID、帖子 ID 等信息
 - 2、调用 ForumEventPublisher 的 notify(e)方法，将事件对象 e 传入
 - 3、notify(e)遍历它维护的观察者列表，依次调用每个观察者的 handle(e)方法
 - 4、MessageObserver、StatisticsObserver、AuditLogObserver 各自收到并处理该事件
- 伪代码可以表示如下：

```
// 发帖业务模块构建事件并触发
ForumEvent event = new ForumEvent("发帖", userId, postId);
publisher.notify(event);

// 发布者内部执行通知逻辑
public void notify(ForumEvent e) {
    for (ForumEventObserver observer : observers) {
        observer.handle(e);    // 每个观察者按自己的方式处理
    }
}
```

六、优点

- (1) 降低耦合：发布者只依赖抽象的观察者接口，不依赖具体实现类。
- (2) 扩展性好：以后若需要新增违规检测等其他功能，只需新增一个观察者类并注册进事件发布者即可，不用改动原有业务代码。
- (3) 符合面向接口编程：各模块各司其职，界限分明，便于分工和后期维护。

综上，观察者模式适合用于论坛事件通知场景，可以将事件触发与事件处理分离，提高系统的可维护性和可扩展性。